

Full length article

A multi-source log semantic analysis-based attack investigation approach

Yubo Song^{a,b,*}, Kanghui Wang^{a,b}, Xin Sun^c, Zhongyuan Qin^a, Hua Dai^c, Weiwei Chen^{a,b},
Bang Lv^c, Jiaqi Chen^a

^a School of Cyber Science and Engineering, Southeast University, Nanjing, China

^b Purple Mountain Laboratories, Nanjing, China

^c State Grid Zhejiang Electric Power Co., Ltd. Research Institute, Hangzhou, China

ARTICLE INFO

Keywords:

Advanced persistent threat

Attack investigation

Provenance graph

Data provenance

ABSTRACT

As Advanced Persistent Threats (APT) become increasingly complex and destructive, security analysts often use log data for performing attack investigation. Existing approaches based on single-source logs fail to capture the causal dependencies between complex attack behaviors. We propose a novel attack investigation approach based on the semantic analysis of multi-source logs. This approach constructs a provenance graph that integrates both application and operating system logs, which can reduce the false positive rate in the attack investigation. Given the substantial size of the graph generated from multi-source logs, we reduce its complexity by merging repeated log events, deleting unreachable nodes, and removing temporary file nodes. To resolve the issue of lacking explicit objectives in current attack investigation approaches, we introduce a new multi-stage investigation approach that enhances the speed of attack investigation. This approach divides an intrusion process into seven distinct attack stages and use a graph pattern matching algorithm to match attack subgraphs belonging to specific attack stages with the provenance graph. This results in an intrusion process composed of attack subgraphs representing individual stages. Experimental results demonstrate that our attack investigation approach increases precision by 15.1% and recall by 12.2%. In terms of time efficiency, our approach reduces investigation time by over 60%, with a minimal decrease of less than 2% in the F1 score.

1. Introduction

Advanced Persistent Threats (APT) (Navarro et al., 2018), as a typical representative of modern complex network attacks, are highly complex and have a long attack cycle. APT attacks usually have multiple stages, including initial access, maintaining authority, lateral movement, external network penetration, etc.¹, and different techniques and strategies may be used in each stage. Although existing intrusion detection systems can identify some attack behaviors, due to the complexity of APT attacks, it is often difficult to build a complete attack picture based on a single security incident. Therefore, security analysts need to conduct attack investigation to fully understand the cause-and-effect relationship between the attacker's attack behaviors and the various links involved in the intrusion process, so as to capture the attack path and effectively block or mitigate the attack.

Logs as primary data recording system and application operations, can capture and document key information in system activities in real time, and allow users to retrospectively trace the entire sequence and causality of events in chronological order. This capability aids

analysts in quickly understanding the sequence and scope of an attack, thereby enabling the formulation of effective defense strategies. With ongoing improvements in kernel-level monitoring frameworks (Gehani and Tariq, 2012; Pohly et al., 2012), existing attack investigation approaches increasingly utilize logs to construct provenance graphs for causal analysis of attack behaviors. Provenance graphs, which are directed acyclic graphs, display data causality with nodes representing system and application entities and directed edges representing interactions between entities, such as process file reads and writes. However, such kernel-level monitoring frameworks generate vast amounts of log data (Ma et al., 2018), leading to coarsely granulated provenance graphs. These logs treat applications as monolithic entities, failing to differentiate between entities within applications, causing interactions generated by different threads to be collectively attributed to the application, thus creating numerous false dependencies (Inam et al., 2023). This issue, known as the dependency explosion problem (Lee et al., 2013; Tang et al., 2018; Xu et al., 2016), complicates security analyses and reduces the effectiveness of current attack investigation

* Corresponding author at: School of Cyber Science and Engineering, Southeast University, Nanjing, China.

E-mail addresses: songyubo@seu.edu.cn (Y. Song), wangkanghui@seu.edu.cn (K. Wang), sun_xin@zj.sgcc.com.cn (X. Sun), zyqin@seu.edu.cn (Z. Qin), dai_hua@zj.sgcc.com.cn (H. Dai), chenweiwei@seu.edu.cn (W. Chen), lv_bang@zj.sgcc.com.cn (B. Lv), chenjq@seu.edu.cn (J. Chen).

¹ <https://attack.mitre.org>

approaches. To address the problem of dependency explosion, Lee et al. (2013) employed reverse engineering to decompose long-running processes into multiple execution units and detected the causal relationships between these units through binary analysis, thus enabling the construction of fine-grained provenance graphs. Similarly, Ma et al. (2017) significantly reduced spatial overhead by adding annotations to the source code of executable files to identify execution units. However, the approach of obtaining and modifying the source code or executable files of programs compromises the integrity of the original programs, making it less feasible in practical applications.

To address the issues outlined above, we introduce a novel attack investigation approach that differs from existing approaches by integrating application logs with operating system logs to construct a provenance graph. This integration enhances the granularity of nodes within the provenance graph. We further optimize the initial provenance graph to reduce its complexity, thereby enhancing the accuracy of attack investigation. Moreover, our approach incorporates a multi-stage attack investigation approach that divides an intrusion process into several specific stages, assisting security analysts in clearly identifying the objectives of investigation. This approach utilizes expert knowledge from threat reports to generate attack subgraphs and employs graph pattern matching algorithms to align subgraphs from specific attack stages with the provenance graph, thereby aiding analysts in quickly analyzing causal relationships between attack behaviors within an intrusion process and speeding up the attack investigation. In summary, this paper makes the following contributions:

- We proposed a provenance graph construction method that integrates application logs and operating system logs. Initially, by analyzing the characteristics of application logs, specific log event fields are extracted. Application logs and operating system logs are then represented in a uniform log event format, and by refining entities, the granularity of nodes in the provenance graph is enhanced. Subsequently, entities and their causal relationships are extracted from the unified log events that merge the semantics of application and system logs. These entities, linked causally to existing detection points and arranged in reverse chronological order, are added to the provenance graph to form the initial provenance graph.
- We designed three optimization rules for the generated initial provenance graph. By merging repeated log events, deleting unreachable nodes, and removing temporary file nodes, an optimized provenance graph is obtained. This reduces the scale of the initial provenance graph without affecting its main structure.
- We proposed a multi-stage attack investigation method to accelerate the speed of attack investigations. First, based on the characteristic multi-stage and interconnected nature of network intrusions, and integrating MITRE ATT&CK and kill chain knowledge, the entire intrusion process is divided into seven stages for staged provenance analysis. Additionally, expert knowledge from threat reports is incorporated, transforming these reports into attack subgraphs representing known attack semantics. These attack subgraphs then serve as the targets for a graph neural network-based graph pattern matching algorithm. This approach associates the unknown behavioral semantics in the provenance graph with known attack semantics from threat reports. By combining the seven stages of an intrusion process and their sequential order, semantic associations between stages are established, ultimately yielding an intrusion process composed of attack subgraphs.

2. Related work

Existing approaches for constructing provenance graphs are primarily divided into two categories: system-level provenance graph construction and execution unit-level provenance graph construction.

Most current research focuses on system-level provenance graph construction. These system-level provenance graphs are typically created by deeply parsing system calls in user space or kernel space to depict the causal relationships and flow of information between entities within the system. In Windows systems, Event Tracing for Windows (ETW) is used for logging, while Linux systems utilize the built-in auditing tool Linux Audit, which intercepts system calls in user space to track various system events. However, this approach incurs significant runtime overhead. To alleviate this issue, Ji et al. (2016) introduced software debugging's record and replay technology, which reduced operational costs while ensuring the security and completeness of data provenance.

Furthermore, considering the limitations of Linux system-level provenance approaches in monitoring kernel operations, which often miss critical information needed for forensic analysis and attack detection, Pohly et al. (2012) were the first to propose intercepting system calls within kernel space, thereby enabling comprehensive collection of system log data. Building on this, Bates et al. (2015) introduced Provmon, which added richer semantic information to system entities, such as file versions, remote IP addresses, and port numbers of network events. However, since both the approaches of Pohly et al. (2012) and Bates et al. (2015) were designed for older kernel versions, they are not directly operational on current Linux systems. To address this issue, Pasquier et al. (2018) developed CamQuery, which utilizes the latest kernel features combined with Linux security modules and NetFilters to capture log data within kernel space.

Existing attack investigation approaches can be categorized based on the granularity of detection into the following types: fine-grained attack investigation approaches focusing on nodes and edges in the provenance graph, coarse-grained approaches based on subgraphs, and approaches that lie between these two, focusing on paths or sequences.

Fine-grained attack investigation approaches aimed to detect whether interactions between nodes in the provenance graph were abnormal. For instance, Zengy et al. (2022) utilized a knowledge graph and a GNN-based recommendation system to detect malicious interactions between nodes, while Han et al. (2021) employed node embedding and autoencoder models to determine if a process derived from a software installation graph was malicious. Although fine-grained approaches offered the highest precision in detection, they typically lacked a comprehensive understanding of contextual information and were time-consuming and labor-intensive. Ding et al. (2023), recognizing that log text and provenance graphs were different representations of the same data that provided similar information, opted to use log files as input and predicted entities related to attacks using the BERT model. This approach maintained fine-grained attack investigation while preserving an understanding of the context. Cheng et al. (2024) employed a novel graph neural network-based encoder-decoder architecture to learn the temporal evolution of structural changes in provenance graphs, thereby quantifying the degree of anomaly for each system event. Subsequently, they reconstructed attack activities based on this fine-grained information. Rehman et al. (2024) employed word2vec (Rong, 2014) and GNN to generate semantic and contextual embeddings, respectively, and constructed a lightweight classifier model that utilizes stored embeddings for real-time anomaly detection.

Coarse-grained attack investigation approaches aim to detect attack subgraphs. For instance, Han et al. (2020) proposed Unicorn, which uses sliding time windows to segment logs and construct evolving subgraphs. Kapoor et al. (2021) developed ProvGem, which captures contextual information of nodes through multiple embeddings and classifies graphs based on aggregated node embeddings using supervised learning. Yang et al. (2023) introduced ProGrapher, which employs graph2vec (Narayanan et al., 2017) to calculate graph embeddings and uses unsupervised approaches to detect anomalous snapshots. Li and Chen (2024) combined provenance graphs with process context information, extracting activity features of processes from the provenance graphs and employing a CNN-BiLSTM (Sinha and Manollas, 2020) model to detect intrusion activities. While coarse-grained approaches

are faster, their precision is lower, requiring security analysts to invest considerable effort in pinpointing malicious nodes and attack events within the attack subgraphs.

Path or sequence-based attack investigation approaches, which offer a granularity between the aforementioned types, focus on detecting anomalous paths or sequences in the provenance graph. Wang et al. (2020) introduced ProvDetector, which utilizes the doc2vec (Le and Mikolov, 2014) word embedding model to convert execution paths into vectors for cluster analysis, identifying abnormal program execution paths. Alsaheel et al. (2021) proposed ATLAS, which extracts sequences of attack behaviors and normal behaviors from the provenance graph, vectorizes these sequences, and uses them as input to train an LSTM model. This model learns the sequential patterns of both attack and normal behaviors, achieving high accuracy and recall rates in its results. Ying et al. (2024) optimized the process of attack investigation through semantic tracking and path analysis. It was built on a dual-stage framework that utilized provenance graphs to simplify the investigation process, focusing on the creation of suspicious semantic graphs and conducting path-level contextual analysis. Yue et al. (2024) proposed a new attack investigation approach based on attack intent-driven and sequence-based learning. This approach constructed sequences of attack and non-attack events from the provenance graph and then used a trained tag-sequence-based semantic model to reconstruct attack scenarios.

3. Preliminaries

3.1. Provenance graph

Recent literature has utilized the concept of data provenance. Compared with manually discovering attack traces from raw logs, data provenance is to build a log provenance graph through a provenance-based system. In these provenance graphs, all system entities from the logs are treated as vertices, and causal relationships between entities are treated as edges. Log events, which represent interactions between entities, are characterized by four main attributes: the subject S performing the action, the object O affected by the action, the time T of the event, and the type (P) of the operation. Thus, an event can be represented as a quadruple: $E = (O \leftarrow S, T, P)$, where S is the subject, O is the object, T is the time, and P is the operation. This causal relationship is also known as a dependency. For two log events $E_i = (O_i \leftarrow S_i, T_i, P_i)$ and $E_j = (O_j \leftarrow S_j, T_j, P_j)$, if $O_i = O_j$, $S_i \neq S_j$, and $T_i < T_j$, then E_i is said to causally influence E_j , or E_j is dependent on E_i . The provenance graph, denoted as $G = (S, O, E)$, is a collection of all entities and log events, where S represents the set of subjects, O represents the set of objects, and E represents the set of events. This graph visualizes entities as nodes and log events as edges between them, where multiple edges between two nodes can represent various types of causal relationships between those entities.

3.2. Graph pattern matching

Graph pattern matching (Mawhirter et al., 2021) is a technique used to identify specific subgraph patterns within large graph datasets. In this process, a *query graph* is defined, which represents the specific pattern that needs to be located within a *target graph*. The graph pattern matching problem is typically defined as follows: given a query graph pattern $Q = (V_Q, E_Q)$, where V_Q is a set of vertices and E_Q is a set of edges with endpoints in V_Q , the task is to find all subgraphs in an input graph G that are isomorphic to Q . Graph pattern matching algorithms are designed to recognize collections of subgraphs that are isomorphic to multiple query patterns. Specifically, if there are two query patterns, Q_1 and Q_2 , the system should be able to identify all subgraphs in the data graph that are isomorphic to each of these patterns separately. To achieve this, algorithms typically generate a matching order that aids in systematically searching for and identifying subgraphs in the data graph that correspond to the query patterns.

3.3. Attack investigation

Attack investigation is a critical technique within the domain of log analysis, aimed at identifying malicious activities from extensive system and network device logs and reconstructing complete intrusion processes. Unlike log anomaly detection, which merely spots irregularities, attack investigation seeks to restore and comprehend the entire attack process, delineating the causal chain behind an attack rather than just pinpointing anomalies. Modern systems involve numerous components and exhibit complex behaviors, often over prolonged attack durations, making attack investigation an exceptionally labor-intensive task. To unearth attack patterns and their underlying causal logic amidst such complex data, contemporary attack investigation approaches increasingly leverage deep learning technologies. These advanced approaches automate the tracing process, thereby more effectively revealing concealed attack patterns and streamlining the task of security analysis in complex network environments.

4. Overview

In this section, we describe the overall architecture of the multi-stage attack investigation framework proposed in this paper, as shown in Fig. 1. Our primary research focuses on the construction of provenance graphs from semantically fused multi-source logs and the multi-stage attack investigation. The provenance graph, enriched with multi-source semantic information, forms the basis for ensuring the correctness and completeness of the attack investigation results, aiming to achieve rapid and accurate attack investigation.

During the construction phase of the provenance graph, due to the single semantic nature and dependency explosion problem inherent in existing provenance graphs built from operating system logs, we propose a approach that integrates both operating system and application logs. Initially, we analyze the characteristics of application logs to extract specific log event fields, representing both application and operating system logs in a uniform log event format. By refining entities, we enhance the granularity of nodes in the provenance graph. Furthermore, we extract entities and their causal relationships from the semantically enriched log events, incorporating entities into the provenance graph in reverse chronological order that have causal links with existing detection points, resulting in an initial provenance graph.

Provenance graphs constructed solely based on operating system-level logs often exhibit coarse granularity. This is because such logs typically treat applications as monolithic entities, unable to distinguish the characteristics of individual entities within the application. This approach results in different threads' interactions being attributed collectively to the entire application, thereby creating numerous inaccurate dependencies within the provenance graph. These imprecise relationships lead to a dependency explosion, where the size and complexity of the provenance graph drastically increase. To address this issue and prevent an overly large provenance graph due to the integration of application logs, we propose three optimization rules. These rules include *merging repeated log events*, *deleting unreachable nodes*, and *removing temporary file nodes*. With these measures, we can effectively reduce the size of the initial provenance graph without compromising its structural integrity, making it more streamlined and efficient.

In the multistage attack investigation phase, addressing the challenges of current attack investigation approaches, which often struggle to capture the semantic nuances of attack behaviors and lack clear objectives during the provenance process, we propose a semantically associated multistage attack investigation approach. We extract text summaries and entity-relationships from threat intelligence reports collected from threat intelligence platforms, transforming involved entities such as processes, files, and IP addresses into graph nodes and translating interactions like calls and read-write actions into directed

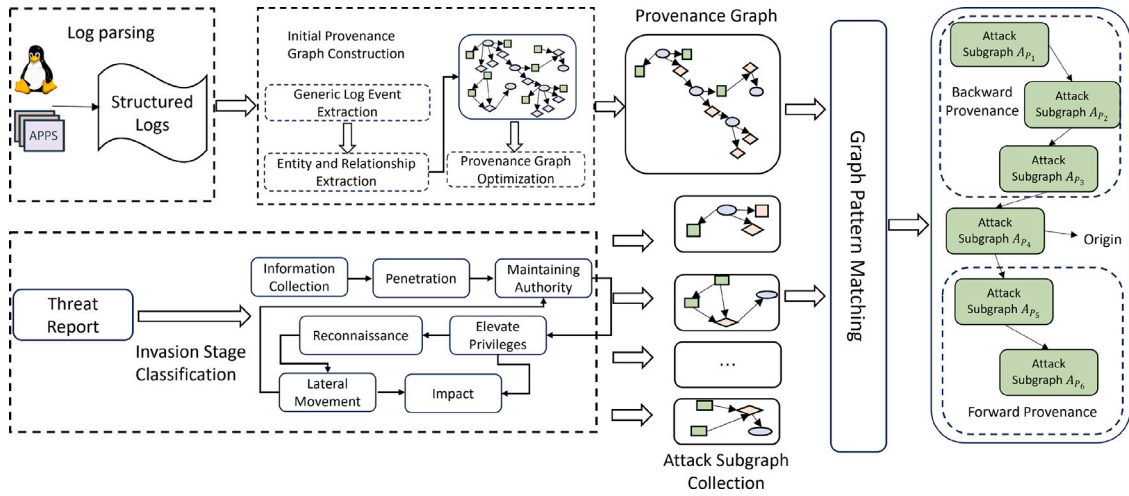


Fig. 1. An overall architecture of the proposed approach in this paper, where A_{P_k} represents the attack subgraph belonging to the attack phase P_k .

```

2023-11-03 12:44:48.514000
UTC- [Main Thread]: V/nsHttp HttpChannelParent::OnStartRequest [this=e35c7c0] Request=15a4982c
2023-11-03 12:44:48.514000
UTC- [Main Thread]: V/nsHttp HttpBaseChannel::SetApplyConversion [this=15a49800 value=0]
2023-11-03 12:44:48.514000
UTC- [Socket Thread]: D/nsSocketTransport calling PR_Read [count=9]
2023-11-03 12:44:48.514000
UTC- [Main Thread]: D/nsHttp nsHttpChannel::GetCacheKey [this=15a49800] Firefox_15a49800 Entity
2023-11-03 12:44:48.514000
UTC- [Socket Thread]: D/nsSocketTransport PR_Read returned [n=9]

```

Fig. 2. An example of entity refinement in Firefox.

edges, representing the attacks from the threat reports as attack subgraphs. We have designed a graph pattern matching algorithm based on Graph Neural Networks (GNNs), which calculates node attribute embeddings based on node type, name, and path, integrating node and neighboring information to derive node embeddings, which are then refined by a globally context-aware attention network to obtain the subgraph embeddings. We incorporate a Neural Tensor Network to evaluate the similarity between attack subgraphs extracted from threat reports and those in the provenance graph. Finally, we divide the entire attack process into seven specific phases, classify attacks into appropriate intrusion phases based on descriptions in the threat reports, and extract the respective phase-specific attack subgraphs. This approach focuses on matching attack subgraphs specific to each phase, thereby narrowing the scope of the provenance and enhancing the speed of attack investigation.

5. System design

In this section, we describe the design details of our proposed approach in detail. As shown in Fig. 1, this approach consists of two parts: provenance graph construction and multi-stage attack investigation. We will further explain these two parts in the following subsections.

5.1. Provenance graph construction

5.1.1. Log parsing

Log parsing is a foundational technique for attack investigation based on logs. We adopt an external tool, LogPPT, as proposed by Le and Zhang (2023), to transform raw unstructured logs into structured formats. Their approach, which involves innovative prompt adjustments to detect keywords and parameters from a small set of labeled log data, has proven both effective and efficient in log parsing.

5.1.2. Initial provenance graph construction

This section aims to integrate application log data with operating system log data into a standard format, enabling the fusion of semantic behaviors at the application layer with underlying operational semantics at the system layer. Because operating system logs capture the overall behavior of the system and application logs record the internal activities of specific programs, the entities involved in these logs differ. Direct merging of these two types of logs to construct a provenance graph is not feasible. By representing both application and system logs in a normalized format, we extract common log events, thus facilitating the integration of user actions within applications and low-level system operations.

Initially, we parse raw logs using the approach described in the *log parsing* subsection, transforming unstructured log data into structured formats and capturing the meaning of each field within the log messages. Subsequently, we extract log events from these messages. Traditional approaches typically extract entities and relationships from logs, thereby converting log statements into log events. Our approach incorporates application layer logs, allowing for further refinement of entities within the logs. Specifically, we utilize *ProcessID*, *ProcessName*, and *SubthreadID* to refine the entities in logs and additionally enrich log events by including fields such as *IP*, *Files*, *Operations*, and *URIs*. For example, we refine the Firefox process entity based on the *SubthreadID* found in the Firefox logs. Using the *this* field in the logs, we differentiate the Firefox entity into two separate entities: *Firefox_e35c7c0* and *Firefox_15a49800*, as shown in Fig. 2.

In addition, to extract general log events, we have designed a standardized format for log entries. Specifically, each log event from application and system logs is parsed into a structured entry comprising an *EventID*, *Timestamp*, *ProcessID*, *SubthreadID*, *ProcessName*, *Operation*, and the resources being accessed, such as *Files*, *IPs*, and *URIs*. If the original logs do not contain information for these fields, their values are set as null. Table 1 shows an example of a generalized log event extracted from Firefox logs.

After extracting a generalized representation of log events from application and system logs, the next step is to construct a provenance graph that integrates log data from both the application and system. Before this, considering that integrating application logs imposes additional computational and storage burdens, we utilize the efficient lossless compression algorithm Snappy² to compress the merged logs. Lossless compression ensures that all original log fields (such as timestamps, process IDs, IP addresses, operation types, etc.) remain

² <https://github.com/google/snappy>

Table 1

An example of a common log event extracted from the Firefox log.

EventID	Timestamp	ProcessID	SubthreadID	ProcessName	IP	File	URI	Operation
1	15:24:48	3165	15f8b100	firefox	58.192.118.142	–	seu.edu.cn	DNS
2	15:24:48	3165	161d5820	firefox	–	page.html	../page.html	Access
3	15:24:48	3165	8a73000	firefox	–	list.html	../list.html	Access

unchanged during both compression and decompression. This algorithm significantly reduces the storage space required for log data, while enabling rapid decompression when needed, thus ensuring the efficiency of the provenance graph construction process.

It is important to note that the provenance graph is constructed offline after an attack is detected. The motivation for constructing a provenance graph is to aid security analysts in identifying suspicious behavior or entities within the system, such as new or modified files or dubious processes. The provenance graph is built based on such detection points, showing all entities and events causally affecting the state of the detection points. We read the logs in reverse chronological order to ensure that the most recently generated logs are read first, and older logs later. We initialize the provenance graph G_0 to contain only entities O_0 and S_0 , where entities O_0 and S_0 form the most recent log event $E_0 = \{O_0 \leftarrow S_0\}$. For each subsequent log event read, if entity O_i is already in the graph, it implies that this log event is relevant to the provenance graph we are analyzing. Further, if another entity S_i from this log event is not yet in the graph, we add entity S_i and establish a new directed edge between O_i and S_i , representing a dependency between these two entities. This process is repeated until the updated provenance graph G is formed. The specific process of constructing the provenance graph is illustrated in Algorithm 1.

Algorithm 1 Provenance Graph Construction

Require: Log dataset L , Log event $E_i = \{O_i \leftarrow S_i\}$, Initial Provenance Graph G_0

Ensure: Updated Provenance Graph G

```

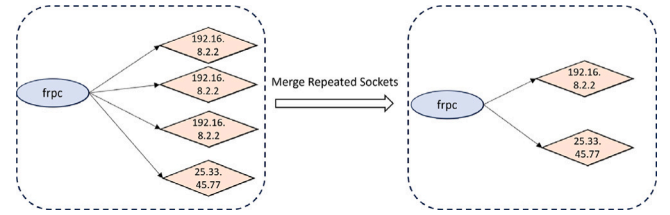
1: for each event  $E_i$  in log  $L$  do
2:   for each object  $O_i$  in graph  $G$  do
3:     if  $E_i$  affects  $O_i$  by the time threshold for  $O_i$  then
4:       if  $S_i$  not in graph  $G$  then
5:         add  $S_i$  to graph  $G$ 
6:         set time threshold for  $S_i$  to time of  $E_i$ 
7:       end if
8:       add edge from  $S_i$  to  $O_i$  to graph  $G$ 
9:     end if
10:  end for
11: end for
12: return  $G$ 

```

5.1.3. Provenance graph optimization

Due to the large scale of logs involving numerous entities and relationships, the provenance graph constructed from these logs is also quite large, posing significant challenges for subsequent analysis. Therefore, this section establishes three optimization rules for the initial provenance graph to reduce its complexity effectively by merging repeated log events, deleting unreachable graph nodes, and removing temporary file nodes.

Merging Repeated Log Events. If certain nodes and edges represent the same type of events, and such events occur repeatedly within a short timeframe, we can merge these nodes and edges to simplify the repeated events into a single event. This approach is taken because these repeated events typically represent continuous access to the same resource or are part of a sequence of continuous actions, which can be regarded as a single event in the provenance graph. For instance, if a process makes multiple socket connections to the same IP address, the initial provenance graph might depict these connections with multiple nodes and edges, as shown in Fig. 3. We can merge these identical socket nodes into one node and consolidate the corresponding edges

**Fig. 3.** An example of merging identical sockets.

into a single edge, retaining the earliest timestamp from the events as the timestamp for the merged event. By merging the repeated events in the initial provenance graph, we significantly reduce the number of nodes, thereby simplifying the structure of the entire provenance graph and easing the analysis workload for security personnel. Moreover, merging these nodes and edges does not affect the causal analysis of the provenance graph.

Deleting Unreachable Nodes. When analyzing a provenance graph starting from a certain detection point, we only focus on the graph nodes and edges that have a causal relationship with this detection point. Therefore, we can remove the unreachable nodes and edges in the provenance graph that have no causal link to the detection point, thereby reducing the size of the provenance graph. Additionally, considering the diversity of nodes in the provenance graph, we implement different strategies for nodes represented by different types of entities during deletion. Specifically, for a process node, it must not only lack a causal relationship with the detection point to be eligible for removal, but it must also have terminated. The reason for this is that if the process is still active, it may still exhibit new behaviors in the future that could have a causal link to the detection point. For other types of nodes, such as files or sockets, they can be removed as long as they do not have a causal relationship with the detection point, without losing any current or future information related to the detection point.

Removing Temporary File Nodes. Logs often contain a large number of temporary files, which have very short lifecycles and are quickly deleted after use. For example, temporary files in Linux systems are typically used to store temporary data generated during program execution, including caches and temporary logs, or as a means of inter-process communication, and are usually located in the system's temporary directories such as `/tmp` or `/var/tmp`. Firefox temporary files primarily store browser cache data, session restoration information, and temporary files from downloads. These temporary files usually exist for a very short period, only interact with the single process that created them, and do not participate in broader system interactions. Therefore, in the context of attack investigation, these files do not introduce any explicit information flow. Based on these characteristics, we can safely remove nodes associated with these temporary files from the provenance graph, a process that neither results in the loss of critical analytical data nor affects the integrity and accuracy of the provenance graph and attack analysis.

These optimization rules effectively reduce the complexity of the initial provenance graph while preserving essential information, allowing security analysts to focus more on the nodes and events that are significant for security analysis, thus enhancing both the efficiency and accuracy of the overall analysis.

5.2. Attack investigation approach

Existing approaches of building provenance graphs based on logs for attack investigation exhibit two deficiencies:

- Attack investigation process takes a long time. Due to the lack of clear objectives in the investigation process, exhaustive searches are typically employed to locate all nodes connected to the trace origin within the provenance graph, and to determine whether these nodes are part of an intrusion event. This approach of exhaustive search overlooks the semantic associations between different attack behaviors throughout the intrusion event, lacks purposefulness, and results in a blind and time-consuming search process.
- Difficult to discover semantics at the attack behavior level. Node-based attack investigation primarily analyzes the attributes or states of nodes, focusing on the relationships between nodes at a fine granularity. However, it lacks contextual and relational understanding, easily overlooking the semantic links between various attack behaviors, rendering the final reconstructed intrusion process lacking in interpretability.

Simultaneously, we have observed that a successful network intrusion is often complex and exhibits the following two characteristics:

- The entire intrusion process consists of multiple stages. The process can be summarized in sequence as follows: *information collection*, *penetration*, *maintaining authority*, *privilege escalation*, *internal reconnaissance*, *lateral movement*, and *impact*, with each stage involving various specific attack behaviors. For instance, in one network intrusion, the penetration stage involved SQL injection and Shiro deserialization attacks, while the maintaining authority stage utilized Dirty COW privilege escalation and SUID exploitation attacks.
- There is a fixed sequence between each two stages of the intrusion process. The next stage of intrusion can only commence after successfully implementing the attacks in the previous stage. For example, if attackers intend to transfer important data files from the victim's machine to a Command & Control (C&C) server (corresponding to the *impact* stage), they must first gain access to the victim's machine through some means (corresponding to the *penetration* stage).

In this section, we address the issue of extensive time consumption due to the lack of clear objectives in the attack investigation in Section 5.2.1. Referring to the MITRE ATT&CK framework,³ and the kill chain knowledge, we divide the entire intrusion process into seven stages. Based on MITRE's Tactics, Techniques, and Procedures (TTPs) knowledge base⁴ we map the semantics of attack behaviors to the respective intrusion stages. Furthermore, we categorize threat reports into these pre-defined seven intrusion stages based on the attack descriptions within the reports, and extract attack subgraphs corresponding to specific intrusion stages from them. In Section 5.2.2, we design a graph pattern matching algorithm based on graph neural networks, using the attack subgraphs as matching targets, and aligning the unknown behavior semantics in the provenance graph with the known attack semantics represented by the attack subgraphs. In Section 5.2.3, we combine the seven stages of the intrusion process and the sequence between these stages to achieve semantic linkage between the stages, ultimately producing an intrusion process composed of elements formed from attack subgraphs.

5.2.1. Attack subgraph extraction

Before extracting the attack subgraphs, we first need to obtain specific descriptions of the attack from threat reports. These reports are widely distributed across security blogs, cybersecurity forums, and major threat intelligence communities. The descriptions of attacks in these reports are presented not only in natural language but also in structured and unstructured standard formats such as OpenIOC,⁵ STIX, and MISP. The main content includes key processes, files, and other entities involved in the attack process, observed suspicious operations, and their interconnections. Previous work has explored extracting text summaries from unstructured online threat intelligence, extracting entity-action relationships, and automatically categorizing online threat intelligence into the MITRE ATT&CK TTPs matrix. Among them, TTPDrill (Husari et al., 2017), based on natural language processing and information retrieval technologies, extracts attack descriptions from unstructured online threat intelligence and converts them into the formatted version of STIX 2.1, then classifies the attacks according to MITRE ATT&CK TTPs. In this paper, we obtain threat reports from four threat intelligence platforms: Symantec,⁶ Fortinet,⁷ TrendMicro,⁸ and Welivesecurity,⁹ and preprocess them using the existing automated threat report analysis tool, TTPDrill.

A complete network intrusion is often complex and contains multiple stages of intrusion. For each stage, the attacker might employ many different attack approaches from the TTPs, and the purpose of this section is to represent these attack approaches (such as SQL injection, DLL file hijacking attacks) in a causal graph form, to facilitate matching attack behaviors and determining intrusion stages in subsequent attack investigation. The representation of these attack approaches in the provenance graph is referred to as attack subgraphs.

We find that security analysts have already conducted detailed studies on existing network intrusions and have provided detailed textual descriptions of their attack implementation processes in threat reports. Fig. 4 shows an excerpted description of an intrusion, recording the process by which the attacker loads the malicious DLL file `imjputyc.dll` by executing `imjpuex.exe`, which then loads the `imjputyc.dat` file. These textual descriptions are essentially the same as log records—they are records of a set of action behaviors, and their format is also very similar, as shown in Fig. 5. For each sentence in the attack description, we can summarize it using an entity-action-entity triplet. Therefore, based on the approach for extracting the provenance graph from logs outlined in Section 5.1.2, we can transform entities such as processes, files, and IP addresses involved in the attack description into nodes in the graph. The action relationships between entities, such as calls and read/write operations, can be converted into directed edges in the graph. In this way, we extract the attack subgraph from the attack description. The specific process is shown in Fig. 4.

It is important to note that each log entry in the log data is discrete, and each log statement is relatively short, which makes it convenient to extract entities and action relationships. Unlike log data, attack descriptions are often long texts that describe each step of the attacker's operations. Therefore, we need to slightly modify the extraction approaches for entities and actions based on this characteristic of attack descriptions. Specifically, we need to treat the entire attack description as a *log event*, with each sentence as a *log entry*, sequentially extracting the actions and entities described in each sentence, and then merging them in order to form the attack subgraph. Finally, we obtain the process descriptions of the attacks from the threat intelligence platforms and extract the attack subgraphs to build a database of attack subgraphs.

⁵ <https://www.mandiant.com/resources/blog/openioc-basics>

⁶ <https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence>

⁷ <https://www.fortinet.com/blog/threat-research>

⁸ https://www.trendmicro.com/en_us/about/threat-research.html

⁹ <https://www.welivesecurity.com/en/threat-reports/>

³ <https://attack.mitre.org/matrices/enterprise/>

⁴ <https://attack.mitre.org/tactics/enterprise/>

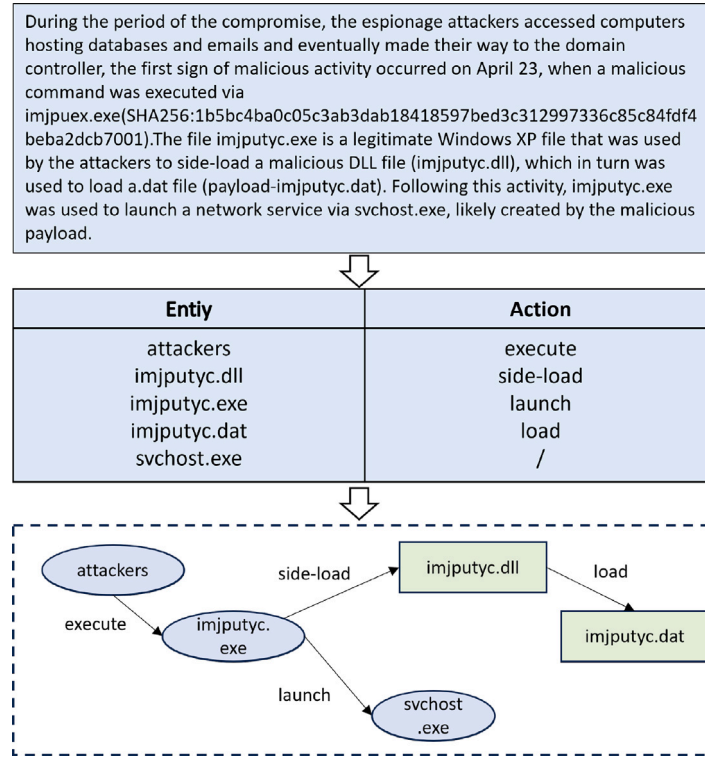


Fig. 4. Attack subgraph extraction process.

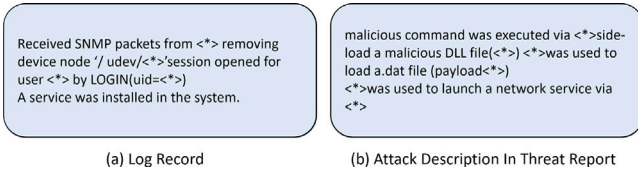


Fig. 5. Comparison of log records and attack descriptions.

5.2.2. Graph pattern matching algorithm

Previous studies (Liu et al., 2023; Mawhirter and Wu, 2019; Chen et al., 2020; Han et al., 2019; Mawhirter et al., 2021) have explored how to search for small graphs that match specific patterns within large graphs. However, these works, not being designed for log attack investigation tasks, primarily focus on exact subgraph matching in isomorphic, undirected graphs. They do not consider the flow of information and causal relationships between nodes, making them unsuitable for direct application in large-scale, heterogeneous, directed provenance graphs. Based on these considerations, we have designed a graph pattern matching algorithm tailored for attack investigation tasks. This algorithm is built upon existing graph neural networks and includes modules for attribute embedding, node embedding, and subgraph embedding.

Attribute Embedding. Since the attack subgraphs are heterogeneous, the nodes in the graph represent different types of entities, such as process entities and file entities. Each type of entity has multiple instances, such as instances of the process entity `svchost.exe` and `chrome.exe`, each with its own process number, process name, and path, as shown in Table 2. In the attribute embedding network, our goal is to combine the semantic information of each node instance to obtain a feature vector for each node. Specifically, we convert each node's attributes into embedding vectors. By aggregating the embedding vectors of the node attributes, we ultimately obtain an embedding vector that represents each node.

In calculating the embedding vectors for node attributes, we view the paths in the provenance graph as sentences in natural language (Wang et al., 2020). We treat nodes representing entities and their attributes as nouns within these sentences, and edges representing causal relationships as verbs. Using the word2vec model (Rong, 2014), we convert each word in the sentence into a vector. Through the word2vec model, we learn the semantics of each attribute and transform them into word vectors. To represent the overall semantics of a node, we aggregate the attributes of the node to obtain the node's embedding vector e_n^0 :

$$e_n^0 = \sum_{i \in A_n} a_i v_i \quad (1)$$

where A_n represents all attributes of node n , i is one of the attributes, v_i is the embedding vector of attribute i , and a_i is the weight of attribute i . Since each attribute of the node represents different meanings and may contribute differently to the overall semantics, it is necessary to assign different weights to each attribute. We use TF-IDF (Kaiser and Ali, 2018) to calculate the weight a_i for each attribute.

Node Embedding. In the attack subgraph, the relationships between a node and its neighboring nodes can provide crucial clues about its behavior and potential maliciousness. Therefore, we aggregate the neighbor information into the node's embedding value to capture the context and relationships of the node within the graph. Since the attack subgraph is constructed based on expert knowledge and generally contains no noise, and the size of the graph is relatively small, we refer to prior work SimGNN (Bai et al., 2019) and use Graph Convolutional Networks (GCN) to compute node embeddings for the attack subgraph.

However, provenance graphs built from logs are often large and contain redundant nodes, making the semantic contribution of each neighbor node vary. Therefore, we refer to DeepHunter (Wei et al., 2021) and incorporate an attention aggregation layer in the GNN, employing an attention mechanism to learn the semantic weights of each neighbor node, more accurately capturing the target node's neighbor information. In the attention aggregation layer, an initial embedding vector e_n^0 for node n and its neighbors are weighted through an attention

Table 2
Typical properties and relationships of nodes.

Node type	Property	Relationship	Shapes in the graph
File	File Name, Path, User	read, write, exec	rectangle
Process	PID, Process Name, Path, User	start, end	oval
Socket	Source IP, Destination IP, Source Port, Destination Port, Protocol	create, connect	rhombus

mechanism. These weights determine the contribution of each neighbor node during the update of node n 's embedding vector. This process is iterative and continues for l iterations. After l rounds of attention aggregation, all intermediate embedding vectors $e_n^0, e_n^1, \dots, e_n^l$ of node n are integrated using a layer-wise dense connection aggregator (Wang et al., 2019), outputting the final embedding vector e_n^{l+1} .

Subgraph Embedding. In the subgraph embedding section, we use node embeddings to generate low-dimensional embedding vectors for both the attack subgraph and the provenance graph. When calculating the weights for different node embeddings, we adopted the Global Context-Aware Attention network proposed by SimGNN (Bai et al., 2019), which allows us to weight the contributions of different nodes in the subgraph embedding pair. Nodes similar to the global context are assigned greater weights, meaning that such node embeddings contribute more to the subgraph embedding. The following formula elucidates the subgraph embedding approach we used:

$$V_G = \sum_{n=1}^N \sigma(e_n^T c) e_n \quad (2)$$

$$= \sum_{n=1}^N \sigma \left(e_n^T \tanh \left(\left(\frac{1}{N} \sum_{m=1}^N e_m \right) W_1 \right) \right) e_n$$

where V_G is the graph embedding value, $\sigma(\cdot)$ is the sigmoid function, $\tanh(\cdot)$ is a nonlinear activation function, and W_1 is a learnable weight matrix.

Similarity Calculation. Simply calculating the dot product of two graph embeddings may not sufficiently reflect their interrelationship. Therefore, when calculating the similarity between two subgraphs, we use Neural Tensor Networks (NTN) (Socher et al., 2013) to model the relationship between the two graph embeddings:

$$g(V_i, V_j) = \tanh \left(V_i^T W_2^{[1:K]} V_j + E \left[\begin{pmatrix} V_i \\ V_j \end{pmatrix} \right] + b \right) \quad (3)$$

where $W_2^{[1:K]} \in \mathbb{R}^{D \times D \times K}$ is a $D \times D \times K$ dimensional weight tensor, the notation $[1 : K]$ represents indices from 1 to K , used to indicate multiple components of the weight tensor, $E \in \mathbb{R}^{K \times 2D}$ is a $K \times 2D$ dimensional weight matrix, and $b \in \mathbb{R}^K$ is a bias vector in $\mathbb{R}^K$ space. K is a hyperparameter used by the model to generate a similarity score for each pair of graph embeddings.

After initially calculating the similarity between the two graph embeddings using a neural tensor network, we use a standard multilayer fully connected neural network to gradually reduce the vector dimension of the similarity score, ultimately obtaining a low-dimensional vector $\hat{s}_{ij} \in \mathbb{R}$, representing the final similarity between the two graph embeddings. Additionally, we use mean squared error to compute the loss function, as follows:

$$\mathcal{L} = \frac{1}{|D|} \sum_{(i,j) \in D} (\hat{s}_{ij} - s(G_i, G_j))^2 \quad (4)$$

where D is the set of training graph pairs, and $s(G_i, G_j)$ is the true similarity between G_i and G_j .

5.2.3. Multi-stage attack investigation

A complete intrusion typically involves multiple closely connected stages. Based on this characteristic, our attack investigation strategy begins by identifying the specific stage of the investigation starting point within the entire attack process, thereby limiting the scope of provenance to this particular stage. We then use graph matching algorithm within this scope to search for potential attacks in the provenance graph. Unlike other graph matching-based provenance approaches, our

approach only matches subgraphs specific to certain intrusion stages, eliminating the need for comprehensive matching against all potential attack subgraphs.

Our attack investigation strategy is divided into two key phases: backward provenance and forward provenance. The backward provenance phase traces back in time to find the initial entry point of the intrusion; the forward provenance phase, on the other hand, proceeds along the timeline to discover potential subsequent attack activities. Before starting the provenance, we extract attack descriptions from an open-source network threat report repository and classify them into seven stages based on the ATT&CK framework TTPs: information collection, external penetration, maintaining privileges, privilege escalation, internal reconnaissance, lateral movement, and impact. Additionally, we extract the corresponding attack subgraphs for each stage from the reports. The specific attack investigation process includes the following steps:

- Step 1 During attack investigation, a given suspected malicious node is typically taken as the starting point G_n for the provenance. Based on the attack stage P_n to which this provenance starting point belongs, we first initiate backward provenance to try to find the entry point of this intrusion. We take the set of attack subgraphs A_{n-1} in stage P_{n-1} as the target to match. For all attack subgraphs A_{n-1}^k in the set A_{n-1} , we use the graph matching algorithm described in Section 5.2.2 to search for matching subgraphs G_{n-1} in the provenance graph constructed in Section 5.1.
- Step 2 After step 1, we have backtracked from the provenance starting point in the provenance graph to find the previous step of the intrusion G_{n-1} . By repeating step 1, we can search for matching subgraphs $G_{n-2}, G_{n-3}, \dots, G_{n-k}$ in the provenance graph for stages $P_{n-2}, P_{n-3}, \dots$, until $n-k = 0$, where we find the matching subgraph G_0 of the initial attack stage. The search process ends, and we have identified the attack behaviors $G_0, G_1, \dots, G_{n-1}$ that occurred before the provenance starting point G_n .
- Step 3 Based on the attack stage P_n of this provenance starting point, we initiate forward provenance to try to find if there are subsequent attack behaviors. Similarly to backward provenance, the first step is to go through the attack subgraph matching process described in step 1 to obtain the matching subgraph G_{n+1} . The difference is that the matching target in the forward provenance phase is the set of attack subgraphs A_{n+1} in stage P_{n+1} . By repeating step (1), we can find the attack behaviors $G_{n+1}, G_{n+2}, \dots, G_{n+m}$ that occurred after the provenance starting point G_n .
- Step 4 By merging the results of backward provenance and forward provenance, we obtain the complete process of this attack $G_0, G_1, \dots, G_{n-1}, G_n, G_{n+1}, \dots, G_{n+m}$. The specific process is shown in Algorithm 2.

After the aforementioned steps, we have avoided the issue of blind matching caused by unclear provenance targets. By clearly defining the provenance targets, we have significantly reduced the analysis time.

6. Evaluation

6.1. Evaluation preparation

To validate the effectiveness of the provenance graph construction approach and attack investigation approach proposed in this paper,

Algorithm 2 Fast Intrusion Tracing Algorithm

Require: Origin G_n , Attack Subgraph Collection A , Attack Phase Collection P , Provenance Graph T

Ensure: $G = \{G_0, G_1, \dots, G_{n-1}, G_n, G_{n+1}, \dots, G_{n+m}\}$

```

1:  $P_n \leftarrow \text{getPhase}(G_n)$ 
2: // Backward Povenance
3: while  $n > 0$  do
4:    $A_{n-1} \leftarrow \text{getAttackGraph}(P_{n-1})$ 
5:   for  $\text{subAttackGraph}$  in  $A_{n-1}$  do
6:     if  $\text{subAttackGraph}$  in  $T$  then
7:        $G_{n-1} \leftarrow \text{subAttackGraph}$ 
8:     end if
9:   end for
10:   $n \leftarrow n - 1$ 
11: end while
12: // Forward Povenance
13: while  $m - n > 0$  do
14:   $A_{n+1} \leftarrow \text{getAttackGraph}(P_{n+1})$ 
15:  for  $\text{subAttackGraph}$  in  $A_{n+1}$  do
16:    if  $\text{subAttackGraph}$  in  $T$  then
17:       $G_{n+1} \leftarrow \text{subAttackGraph}$ 
18:    end if
19:  end for
20:   $n \leftarrow n + 1$ 
21: end while
22: return  $G = \{G_0, G_1, \dots, G_{n-1}, G_n, G_{n+1}, \dots, G_{n+m}\}$ 

```

Table 3

Dataset used to construct the provenance graph for the test phase.

Dataset	Number of attacking entities	Number of normal entities	Size (MB)
D_0	47	28 904	930
D_1	23	19 441	654
D_2	31	21 694	746

we conducted corresponding experiments. Our log collection was performed on the Linux platform, while the construction of provenance graphs and attack subgraphs was carried out on the Windows platform, with the GNN model deployed on the PyTorch platform. Additionally, we utilized the open-source graph computing framework Apache Tinkerpop¹⁰ to construct a PostgreSQL graph database. The implementation of our approach was deployed on a Windows computer equipped with an Intel i7-13700KF CPU @ 3.4 GHz, NVIDIA GeForce RTX 3090, and 64 GB of memory.

We recreated attack behaviors and mimicked normal user actions, using the operating system and application logs generated during this time period as the test dataset for our approach. Since user behavior dominates the dataset with only a small portion being attack behaviors, we predominantly mimic normal user activities, such as browsing the web, watching videos, and using office software. As for the attack behavior data, we first selected a subset of threat reports, and then, based on the descriptions in these reports, randomly chose a time to replicate the attack behavior. This approach ensures that the ratio of attack behavior logs to normal user behavior logs in the dataset is closer to real-world scenarios. Following this methodology, we generated three datasets, D0, D1, and D2, as shown in Table 3. In these datasets, attack entities refer to specific objects or elements that can be clearly identified during the execution of the attack and can only be associated with the attack event itself. We used this criterion to distinguish them from normal entities.

In dataset D0, the attacks performed at each stage of the intrusion include port scanning, SQL injection, modifying the registry, Dirty Cow

privilege escalation (National Vulnerability Database (NVD), 2016), EternalBlue, SSH remote connection, and file modification. In dataset D1, the attacks performed at each stage of the intrusion include port scanning, SQL injection, adding scheduled tasks, Dirty Cow privilege escalation, asset collection, SSH remote connection, establishing a reverse proxy, and modifying webpage content. In dataset D2, the attacks performed at each stage of the intrusion include subdomain enumeration, phishing emails, modifying the registry, SUID exploitation (National Vulnerability Database (NVD), 2021), EternalBlue, account enumeration, and sensitive file leakage.

During the provenance graph construction phase, to validate the role of application logs and the effectiveness of the optimization rules for provenance graphs, we selected attack reports published by threat intelligence platforms (Jiang et al., 2017; Trellix Research Team, 2023; Trend Micro Research Team, 2016), replicated three APT attack scenarios, and simulated normal user operations during non-attack periods. Moreover, to evaluate the spatial and temporal overhead of integrating application logs, we simulated regular user activities such as web browsing, video watching, downloading, and editing files. Simulations also included routine operations executed by security operations personnel in Bash, including SSH remote login, system status checks, service management, user and group management, configuring firewall rules, and viewing log files. Finally, we conducted stress tests on the Web server using the ApacheBench¹¹ tool, simulating multi-user concurrent access to Apache and Nginx servers. When evaluating the role of application layer logs, we primarily used precision and recall as metrics.

In evaluating the effectiveness of provenance graph optimization, we first compare the number of nodes and edges in the graph before and after applying the optimization rules to measure the extent to which the optimization simplifies the graph's scale. However, considering the potential for false negatives during the optimization process (i.e., the deletion of critical information related to the attack behavior), we further conduct experiments using dataset D0. Specifically, for each attack subgraph in D0, we extract and mark the critical paths and critical nodes. We define a critical path as the core sequence of events leading from the attack's origin to its final objective, while a critical node refers to an indispensable entity along these paths (such as key processes, critical files, or vital network connections). Prior to optimizing the provenance graph constructed from dataset D0, we label the critical paths and nodes for all attack subgraphs in D0. To assess the retention of key information in the optimized provenance graph, we define the "Critical Information Retention Rate" (CIRR) as an evaluation metric. Assuming equal weight is assigned to both critical nodes and critical paths, the CIRR is calculated as follows:

$$CIRR = \frac{1}{2} \left(\frac{|N_{\text{after}} \cap N_{\text{critical}}|}{|N_{\text{critical}}|} + \frac{|P_{\text{after}} \cap P_{\text{critical}}|}{|P_{\text{critical}}|} \right) \times 100\% \quad (5)$$

where N_{critical} is the set of all critical nodes labeled before optimization, and N_{after} is the set of nodes in the optimized provenance graph; P_{critical} is the set of all critical paths labeled before optimization, and P_{after} is the set of paths that remain in the optimized graph. In evaluating the additional spatial overhead brought by integrating application logs into the provenance graph, we mainly compared the size of application logs with operating system logs. For the additional temporal overhead, we compared the time taken to generate the provenance graph before and after integrating application logs.

In the attack investigation phase, we selected threat reports from open-source threat intelligence platforms such as Symantec, Fortinet, TrendMicro, and Wivesecurity as datasets for extracting attack subgraphs, as shown in Table 4. These reports detailed the operational procedures of specific attacks. We applied the automated tool TTP-Drill (Husari et al., 2017) to preprocess the threat reports, extracting necessary entities such as processes and files and their relationships to

¹⁰ <https://tinkerpop.apache.org/>

¹¹ <https://httpd.apache.org/docs/2.4/programs/ab.html>

Table 4
Threat Report Data Source Statistics.

Data source name	Quantity (portions)
Symantec Threat Intelligence	28
TrendMicro Security Encyclopedia	26
Fortinet Security Intelligence	21
Welivesecurity Threat Intelligence	11
Total	86

construct attack subgraphs, with manual fine-tuning to reduce noise. We used the publicly available log dataset ATLAS (Alsaheel et al., 2021) to construct the provenance graph and optimized it according to the rules proposed in this paper, using this provenance graph as the basis for training our graph matching model. In addition, we use datasets D1 and D2 as test sets to evaluate the effectiveness of our graph matching algorithm and attack investigation approach.

In the evaluation experiments of the graph matching algorithm, we primarily use two metrics: the time required to generate similarity scores between two graphs and the Mean Squared Error (MSE). The MSE is the average of the squared differences between the calculated similarity scores and the true similarities, defined as follows:

$$MSE = \frac{1}{n} \sum_{i=1}^n (s_i - \hat{s}_i)^2 \quad (6)$$

where s_i is the true similarity and $\hat{s}_i$ is the similarity calculated by the graph matching algorithm. For the true similarity $s(G_i, G_j)$ between G_i and G_j , we first compute the normalized Graph Edit Distance (GED):

$$\text{normalizedGED}(G_i, G_j) = \frac{\text{GED}(G_i, G_j)}{\frac{|G_i| + |G_j|}{2}} \quad (7)$$

where $|G_i|$ represents the number of nodes in graph G_i and $\text{GED}(G_i, G_j)$ represents the graph edit distance between G_i and G_j . Then, we map the similarity scores to the range (0, 1] using the exponential function $\lambda(x) = e^{-x}$:

$$s(G_i, G_j) = e^{-\text{normalizedGED}(G_i, G_j)} \quad (8)$$

Furthermore, we evaluate the overall performance of our proposed attack investigation approach using Precision, Recall, F1-Score, and the time taken.

In terms of model parameter settings, we set the number of layers in the Graph Convolutional Network (GCN) to 3, using ReLU as the activation function. The output dimensions of the 1st, 2nd, and 3rd layers of the GCN are 64, 32, and 16, respectively. In the Neural Tensor Network, we set K to 16 and use four fully connected layers to reduce the dimensionality of the module connection results: from 32 to 16, 16 to 8, 8 to 4, and finally from 4 to 1. During the training phase, we set the batch size to 128, optimize using the Adam algorithm, and fix the initial learning rate at 0.001. We set the number of iterations to 10,000 and select the best model based on the lowest validation loss.

6.2. Evaluation of provenance graph construction approach

6.2.1. The role of application layer logs

To assess the actual impact of integrating application layer logs on the semantic completeness of provenance graphs, we simulated three typical APT attack scenarios based on attack reports published by threat intelligence platforms. During these simulated scenarios, we meticulously recorded the log data produced by the operating system and applications. From these logs, we constructed two types of provenance graphs: a traditional provenance graph primarily based on operating system logs, and a multi-layer semantic integrated provenance graph that combines both operating system and application logs. Subsequently, we evaluated the performance of the classic attack investigation approach ATLAS (Alsaheel et al., 2021) in terms of precision and recall on these two types of provenance graphs, with the

experimental results shown in Table 5.

After comparative analysis, we observe a significant improvement in the performance of the classic attack investigation approach, ATLAS, when application logs are integrated into the provenance graph alongside operating system logs. Specifically, within the framework of traditional provenance graphs, which rely solely on operating system logs, the semantic information provided is relatively limited, making it challenging to fully capture the nuanced behaviors of users at the application layer. This limitation often results in the incorrect classification of entire application entities as hostile when some internal actions might be benign, leading to a higher rate of false positives and an average precision rate of only 0.73 in our simulated attack scenarios. However, in the provenance graphs constructed using the approach proposed in this paper, which integrates application logs, there is a more precise representation of user behaviors at the application layer. This improvement significantly reduces the instances of false positives, thereby enhancing the precision of ATLAS. With the integration of application logs, ATLAS's average precision rate in the provenance graphs increases to 0.84, an improvement of 15.1% over traditional graphs. Additionally, we observed a notable improvement in recall rates. In traditional provenance graphs, ATLAS's average recall was only 0.82. However, in provenance graphs built using our approach, which minimizes erroneous dependencies and refines the granularity of application entities, ATLAS's average recall rose to 0.92, a 12.2% increase over traditional approaches.

From the above experimental data and analysis, we conclude that integrating application logs significantly enhances the semantic accuracy and completeness of provenance graphs. This advancement not only allows the provenance graphs to reflect user behavior at the application level more precisely but also provides a more comprehensive and accurate basis for subsequent log analysis tasks, such as attack investigation. By integrating application logs, we can more accurately capture malicious activities in complex attack scenarios, reduce false positives, and enhance overall analysis efficiency.

6.2.2. Optimization effects of provenance graph

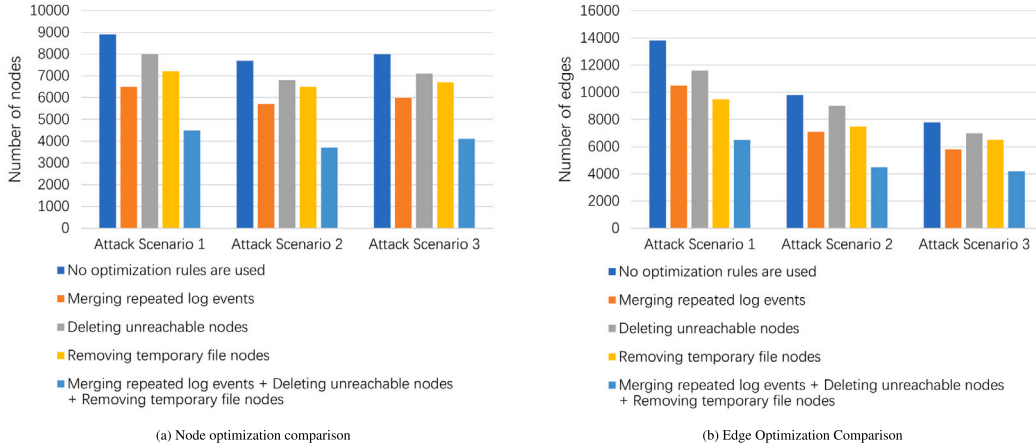
To validate the effectiveness of the three provenance graph optimization rules proposed in this paper, we constructed five different provenance graphs based on log data generated from three simulated APT attack scenarios. These graphs were evaluated based on the number of nodes and edges they contain, which served as metrics to measure the scale of the provenance graphs and to further analyze the effectiveness of the proposed optimization rules. Specifically, we generated five provenance graphs: a baseline graph without any optimization, graphs optimized using the rules for *merging repeated log events*, *deleting unreachable nodes*, and *removing temporary file nodes*, and finally a graph that applied all three optimization rules simultaneously. The number of nodes and edges in these five provenance graphs is shown in Fig. 6.

Based on the comparison of experimental results, we draw the following conclusions: Firstly, without using any optimization rules, the provenance graphs we generated are extremely large in scale. The average number of nodes in the provenance graphs constructed from three attack scenarios is about 8200, with an average of over 11000 edges. This reflects the redundancy and complexity in the original log data, posing challenges for subsequent forensic and analytical tasks. Secondly, after applying the three optimization rules to the baseline provenance graph, we observed a significant reduction in the size of the provenance graphs. Specifically, the rule for *merging repeated log events* resulted in an average reduction of 25.5% in the number of nodes and 23.8% in the number of edges; the *deleting unreachable nodes* rule reduced the number of nodes by an average of 10.1% and the number of edges by 10.0%; the *removing temporary file nodes* rule led to a 15.6% reduction in nodes and a 21.4% reduction in edges. Finally, when all three rules were applied simultaneously to the baseline provenance graph, the number of nodes decreased by 48.9% and the number

Table 5

Comparison of the precision and recall of the classic attack investigation approach ATLAS on the traditional provenance graph (using only operating system logs) and the provenance graph in this paper (operating system logs + application logs).

Attack scenario	Precision		Recall	
	Operating system logs	Operating system logs + application logs	Operating system logs	Operating system logs + application logs
Attack Scenario 1	0.72	0.84	0.82	0.91
Attack Scenario 2	0.73	0.84	0.81	0.92
Attack Scenario 3	0.73	0.83	0.83	0.92

**Fig. 6.** Comparison of provenance graph optimization effects of three rules.

of edges by 48.8%. This is because provenance graphs constructed solely based on operating system logs are coarse-grained. Such logs treat applications as monolithic entities, unable to distinguish between internal entities of the applications. This causes interactions generated by different threads to be collectively considered as interactions of the application, thereby creating numerous false dependencies which ultimately lead to dependency explosion issues. Our experimental results show that the original provenance graphs contained a large amount of noise and redundancy. The provenance graph optimization rules we proposed effectively removed these unnecessary parts. While preserving the main structure of the provenance graph, it reduced the scale of nodes and edges in the initial graph, avoiding redundant computations of information flow during the analysis. This optimized the consumption of computational resources and the time costs throughout the analysis process. This approach significantly reduced the complexity of the graph, focusing it more on essential information, thereby alleviating the dependency explosion problem. Additionally, the streamlined graph facilitates subsequent processing and analysis by security analysts.

Furthermore, to verify whether the optimization rules result in the removal of critical information related to attack behaviors, we denote the provenance graph constructed from dataset D0 as G_{before} . Initially, based on the known attack procedures and behavioral semantics, we label the critical paths and critical nodes for each attack subgraph in D0. For instance, in the case of the Dirty Cow privilege escalation attack subgraph, we label the event chain from executing the Dirty Cow exploit to successfully modifying the `/etc/passwd` file as the critical path, and the associated process nodes and `/etc/passwd` file node as critical nodes. After marking the critical information, we apply the proposed three optimization rules to G_{before} , resulting in the optimized provenance graph G_{after} . We then compare the number of critical nodes and critical paths still present in G_{after} with their baseline values in G_{before} , calculating the Critical Information Retention Rate (CIRR). The detailed process is outlined in Algorithm 3.

The experimental results are shown in Table 6. Before applying the three optimization rules, we marked 163 critical nodes, which include both attack entities and related nodes necessary for understanding the attack process. After optimization, the number of critical nodes decreased to 148, with a critical node retention rate of approximately

Algorithm 3 Critical Information Retention Rate Calculation.

Require: The initial provenance graph G_{before} , The optimized provenance graph G_{after} , A set of attack entities $AttackEntities$

Ensure: CIRR: Critical Information Retention Rate

```

1:  $N_{critical} \leftarrow \emptyset$ 
2:  $P_{critical} \leftarrow \emptyset$ 
3: for each entity  $e$  in  $AttackEntities$  do
4:    $N_{critical} \leftarrow N_{critical} \cup \{e\}$ 
5: end for
6:  $AddCritNodes \leftarrow IdtCritNodes(G_{before}, AttackEntities)$ 
7:  $N_{critical} \leftarrow N_{critical} \cup AddCritNodes$ 
8:  $Subgraphs \leftarrow IdentifySubgraphs(G_{before})$ 
9: for each subgraph  $S$  in  $Subgraphs$  do
10:   $Paths_{in-S} \leftarrow ExtractPaths(S, N_{critical})$ 
11:   $P_{critical} \leftarrow P_{critical} \cup Paths_{in-S}$ 
12: end for
13:  $N_{after} \leftarrow ExtractNodes(G_{after})$ 
14:  $P_{after} \leftarrow ExtractMatchPaths(G_{after}, P_{critical})$ 
15:  $N_{retained} \leftarrow N_{after} \cap N_{critical}$ 
16:  $P_{retained} \leftarrow P_{after} \cap P_{critical}$ 
17:  $NodeRetainRate \leftarrow \frac{|N_{retained}|}{|N_{critical}|}$ 
18:  $PathRetainRate \leftarrow \frac{|P_{retained}|}{|P_{critical}|}$ 
19:  $CIRR \leftarrow \frac{(NodeRetainRate + PathRetainRate)}{2} * 100\%$ 
20: return CIRR

```

90.8%. This indicates that while the optimization rules reduced redundant information, they effectively preserved the nodes that are crucial for understanding the attack process. Prior to optimization, 24 critical paths were marked, representing the core event chains in the attack subgraphs. After optimization, 22 critical paths were retained, yielding a path retention rate of approximately 91.7%, which suggests that the optimization process did not significantly disrupt the coherence and interpretability of the attack behavior chain. The Critical Information Retention Rate (CIRR) was calculated to be approximately 91.25%.

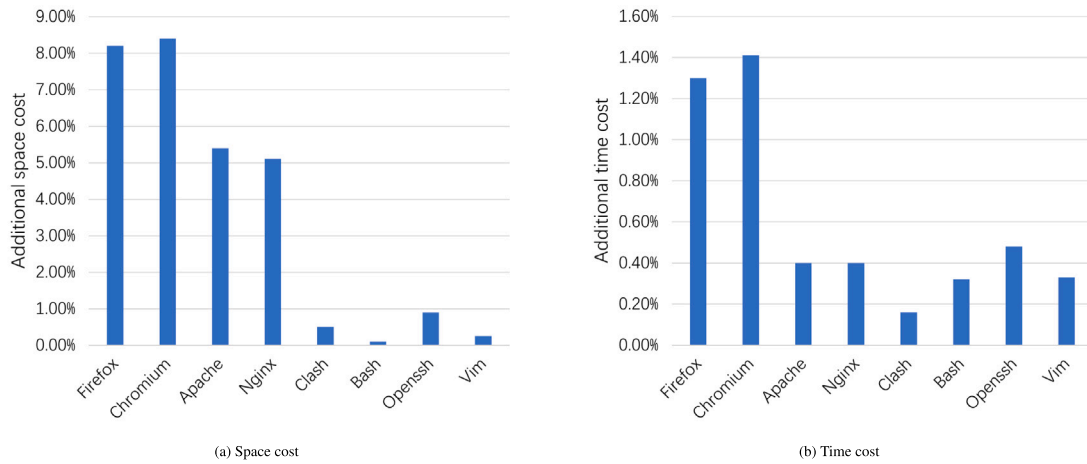


Fig. 7. Comparison of additional space and time overhead caused by eight typical application logs.

Table 6

Comparison of the number of critical nodes and critical paths before and after applying the three optimization rules.

Dataset	Number of critical nodes	Number of critical paths
G_{before}	163	24
G_{after}	148	22

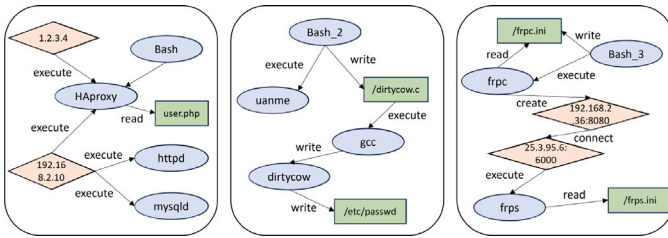


Fig. 8. Three attack subgraphs extracted from the threat report.

However, the three optimization rules did not retain all critical information. When the provenance graph contains multiple attack subgraphs, the critical paths and nodes of these subgraphs may overlap or interfere with one another. For instance, some log events may have different levels of importance in different attack scenarios, leading to the inadvertent removal of key nodes in specific attack subgraphs during the optimization process. Additionally, these three optimization rules were designed to reduce the complexity and storage overhead of the provenance graph. In most cases, these rules effectively eliminate redundant information, thereby improving the efficiency and usability of provenance analysis.

6.2.3. Additional space and time overhead

To test the additional space and time overhead introduced by integrating application layer logs into the provenance graph, we deployed and ran eight widely used applications on a Linux host. These include browser applications FireFox and Chromium, web server applications Apache and Nginx, network proxy application Clash, command line application Bash, remote connection application Openssh, and text editor Vim. Over a 24-hour experimental period, we simulated normal user behaviors, performed a variety of operations, and captured the resulting operating system logs (Linux Audit) as well as logs from various applications.

Regarding extra space overhead, we assessed this by comparing the size of logs generated by these eight applications during the period to the size of logs generated by the operating system. For extra time overhead, we first created a provenance graph using only the operating

system logs recorded during this period, noting the time taken to generate this graph. We then merged the logs from these eight applications into the operating system logs and generated the provenance graph again, recording the time taken. Finally, we compared the time required to generate the provenance graphs before and after merging the application logs. The results are shown in Fig. 7.

From Fig. 7(a), it is evident that the overall extra space overhead from the logs of these eight typical applications was less than 9%, which is within an acceptable range and clearly demonstrates that integrating additional application layer logs has a minimal impact on the space overhead of the provenance graph. The primary contributors to space overhead were browser applications FireFox and Chromium, and web server applications Apache and Nginx, while the other four applications contributed significantly less to the extra space overhead, each under 1%. Fig. 7(b) shows that the extra time overhead for generating the provenance graph due to merging these application logs was also minimal. Although Firefox and Chromium, due to their larger log volumes, contributed relatively higher time overhead, they still remained under 1.5%. Overall, given the benefits provided by integrating application logs, these minor space and time overheads are considered very worthwhile.

6.3. Evaluation of attack investigation approach

6.3.1. Effectiveness of graph matching algorithm

To validate the effectiveness of this graph matching algorithm in identifying attack subgraphs, we collected threat reports for each of the seven stages of the intrusion process, implemented the attacks as described in the reports, and logged the activities during these periods. Fig. 8 displays three attack subgraphs extracted from the threat reports. Subsequently, we transformed the collected logs into provenance graphs as described in Section 5.1 and performed subgraph matching within the provenance graphs. The matching results are shown in Fig. 9.

To evaluate the effectiveness of our attack subgraph matching, we compared the mean square error (MSE) between the true similarity and the model-calculated similarity of attack subgraphs extracted from the threat reports against actual attack behavior graphs in the provenance graphs. The experimental results, as shown in Table 7, demonstrate that all attack subgraphs in datasets D1 and D2 have an MSE of less than 1.4%, proving the effectiveness of our graph matching algorithm.

Furthermore, we compared our attack subgraph matching algorithm with several common graph matching algorithms, including: (1) Beam (Neuhaus et al., 2006), a sub-exponential time GED algorithm based on the A* algorithm. (2) SimpleMean, which calculates graph embeddings by taking a simple average of all node embeddings in the graph. (3) HierarchicalMean (Defferrard et al., 2016), a graph

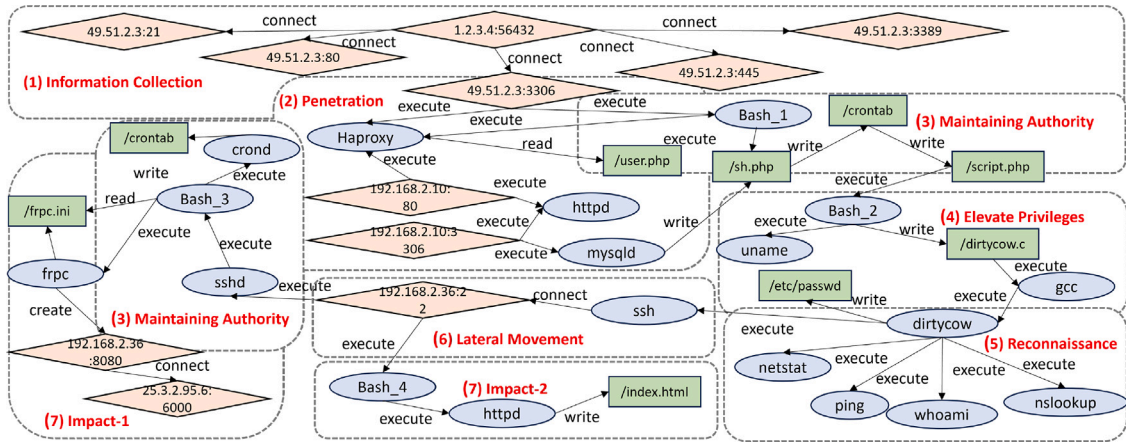


Fig. 9. Attack subgraph matching results of dataset D1.

Table 7

The mean square error between the calculated similarity and the true similarity.

Dataset	Attack	MSE (10^{-2})
Dataset D1	Port Scanning	1.39
	SQL Injection	1.40
	Adding Scheduled Tasks	1.32
	DirtyCOW Privilege Escalation	1.36
	Asset Collection	1.26
	SSH Remote Connection	1.21
	Establish Reverse Proxy	1.29
	Modify Web Page Content	1.17
Dataset D2	Subdomain Collection	1.19
	Phishing Emails	1.27
	Modify Registry	1.21
	SUID Exploitation	1.36
	EternalBlue	1.26
	Account Enumeration	1.34
	Sensitive File Leakage	1.29

coarsening convolutional neural network model that applies global average pooling to aggregate node features into a hierarchical graph representation. (4) AttDegree, which uses the natural logarithm of node degrees as attention weights. (5) AttGlobalContext, which computes attention weights using the global graph context. We evaluated these algorithms on their average MSE and average matching time for attack subgraph matching. The results are shown in Table 8.

Through experimental comparative analysis, we observed that the heuristic algorithm Beam lags behind other algorithms in graph matching tasks. Specifically, Beam's mean squared error (MSE) is significantly higher than that of neural network-based approaches, indicating its inadequacy in graph matching accuracy. Moreover, Beam's graph matching time also far exceeds other approaches, which may be due to its inefficiency in handling large-scale or complex graph structures using the A* search approach. In contrast, our proposed graph neural network-based pattern matching algorithm exhibits superior performance. Specifically, our algorithm not only performs best in terms of MSE but also is highly competitive in computation time, second only to the SimpleMean approach. Our neural network model, by learning the composite features of nodes and their neighborhoods and dynamically weighting the importance of nodes through attention mechanisms, can more accurately capture the underlying connections between nodes in graphs with complex interaction patterns, thereby significantly enhancing the accuracy of graph matching. In contrast, although SimpleMean performs well in terms of graph matching time, its lack of dynamic weight adjustment in processing node features results in suboptimal graph matching outcomes, with its MSE being 51.0% higher than our approach.

Table 8

Performance comparison of our approach with other graph matching algorithms.

Approach	AvgMSE (10^{-3})	AvgTime (seconds)
Beam	10.11	360
SimpleMean	3.12	48
HierarchicalMean	1.62	91
AttDegree	3.05	69
AttGlobalContext	3.13	72
Our Approach	1.53	67

Table 9

Comparison between our approach and other attack investigation approaches.

Dataset	Approach	Precision	Recall	F1-Score	Time costs (seconds)
Dataset D1	ATLAS	0.91	0.91	0.91	120
	AIRTAG	0.92	0.91	0.92	40
	OmegaLog	0.86	0.87	0.86	165
	Our approach	0.91	0.90	0.91	45
Dataset D2	ATLAS	0.92	0.92	0.92	120
	AIRTAG	0.92	0.92	0.92	40
	OmegaLog	0.86	0.88	0.87	180
	Our approach	0.90	0.91	0.91	45

6.3.2. Analysis of multi-stage attack investigation approach

We conducted a comparative analysis with the existing common attack investigation approach ATLAS (Alsaheel et al., 2021), AIRTAG (Ding et al., 2023), and OmegaLog (Hassan et al., 2020) on datasets D1 and D2, analyzing the model's precision, recall, F1 score, and time consumed, to evaluate whether the model achieves the anticipated functional effects. For a uniform comparison standard, we deconstructed the attack subgraphs identified during the attack investigation phase and extracted nodes within them, considering the adjacent nodes of these subgraphs as the identified attack entities and nodes outside the subgraphs as normal entities. The experimental results are shown in Table 9. Through comparative analysis of experimental results, our approach demonstrated outstanding performance. We detailed the analysis of three core performance metrics: precision, recall, and F1 score, which in experiments reached 90.5%, 90.5%, and 91.0%, respectively. Compared to other mainstream attack investigation approaches such as ATLAS, AIRTAG, and OmegaLog, our approach showed only a minor decrease of 1.1%, 1.1%, and 0.5% against ATLAS, and 1.6%, 1.1%, and 1.1% against AIRTAG, but outperformed OmegaLog by 5.0%, 3.3%, and 4.9%, respectively. The results indicate that our approach achieved the expected effectiveness of attack investigation.

Additionally, from the perspective of time efficiency, our approach performed exceptionally well in the time required for attack investigation, saving 62.5% compared to ATLAS and 73.9% compared to OmegaLog. Although our approach took 11.1% more time than AIRTAG,

considering AIRTAG's reliance on the BERT model for deep semantic learning, which requires significant hardware resources and long pre-training times, our approach offers greater time and cost efficiency in practical applications. This is because we have found that network attacks are often implemented in stages and are closely connected between stages, yet existing attack investigation approaches largely overlook this, leading to a considerable amount of time spent searching for non-existent attack behaviors. To address this problem, we pre-divide network attacks into several specific attack stages, focusing on specific stage attack subgraphs to narrow the provenance scope, thereby aiding security analysts in conducting rapid attack investigations. Thus, we can conclude that our attack investigation approach, at the cost of slight precision and recall rates, significantly saves time during the provenance phase.

7. Discussion & limitation

We have introduced a provenance graph construction approach based on semantic fusion, which utilizes the characteristics of application layer logs. By extracting specific log event fields, application layer logs and operating system logs are represented in a common log event format. Entities and their causal relationships are extracted from these common log events to form an initial provenance graph. Through merging repeated log events, deleting unreachable nodes, and removing temporary file nodes, an optimized provenance graph is obtained. Experimental results demonstrate that, compared to traditional provenance graphs, the downstream forensic tasks, such as attack investigation, achieve higher precision and recall on the provenance graph constructed in this study. Additionally, the analysis of the attack investigation approach proposed in this paper shows that it can accurately and rapidly replicate attack processes, assisting security personnel in quickly analyzing the causal relationships between attack behaviors within an attack sequence. Although our logs are collected on the Linux platform, this approach can be easily extended to other platforms like Windows, due to its reliance on log semantic analysis to construct provenance graphs with specific causal relationships.

This paper utilizes both operating system and application logs to build provenance graphs, reflecting user behavior semantics at the operating system and application levels. It is worth noting that our current research approach is primarily based on real and complete log data for provenance analysis. However, as Advanced Persistent Threats (APTs) become increasingly sophisticated, APT attacks may generate false logs or even tamper with existing logs. Consequently, our approach may face certain limitations when APTs intentionally fabricate or corrupt logs to mislead attack investigations. In future research, we will explore potential countermeasures to address APTs' anti-forensic behaviors. In our multi-stage attack investigation approach, we divide the attack process into seven fixed stages using expert knowledge, and extract attack subgraphs from existing threat reports. These subgraphs are used as query standards to search for attacks within the provenance graph. Although this approach significantly reduces the provenance analysis time, the accuracy of the results is largely dependent on the selection of expert knowledge, and it is also challenging to detect unknown or hybrid attacks. Therefore, future work should focus on reducing the reliance on expert knowledge while improving the model's generalization ability to enhance detection capabilities for unknown and hybrid attack patterns.

8. Conclusion

We have proposed an attack investigation approach based on semantic analysis of multi-source logs. Specifically, this approach integrates operating system logs with application logs to construct an initial provenance graph. The initial graph is then further optimized to reduce complexity. Finally, the approach employs a graph pattern matching

algorithm combined with previously defined seven stages of the intrusion process to achieve rapid attack investigation. Our evaluation with real attacks demonstrates that this approach can accurately and swiftly reconstruct the intrusion process and assist security personnel in analyzing its potential causal relationships.

CRedit authorship contribution statement

Yubo Song: Writing – original draft, Supervision, Methodology, Investigation, Data curation, Conceptualization. **Kanghui Wang:** Writing – review & editing, Writing – original draft, Software, Methodology, Investigation. **Xin Sun:** Writing – original draft, Supervision, Software, Investigation, Funding acquisition. **Zhongyuan Qin:** Writing – review & editing, Validation, Project administration. **Hua Dai:** Project administration, Funding acquisition. **Weiwei Chen:** Writing – review & editing, Investigation. **Bang Lv:** Project administration, Funding acquisition. **Jiaqi Chen:** Writing – review & editing, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by State Grid Zhejiang Electric Power Co., Ltd. research project No. 5211DS240002.

Data availability

Data will be made available on request.

References

- Alsaheel, A., Nan, Y., Ma, S., Yu, L., Walkup, G., Celik, Z.B., Zhang, X., Xu, D., 2021. ATLAS: A sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 3005–3022.
- Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., Wang, W., 2019. SimGNN: A neural network approach to fast graph similarity computation. In: Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining. pp. 384–392.
- Bates, A., Tian, D.J., Butler, K.R., Moyer, T., 2015. Trustworthy whole-system provenance for the linux kernel. In: 24th USENIX Security Symposium. USENIX Security 15, pp. 319–334.
- Chen, X., Dathathri, R., Gill, G., Pingali, K., 2020. Pangolin: An efficient and flexible graph mining system on cpu and gpu. Proc. VLDB Endow. 13 (8), 1190–1205.
- Cheng, Z., Lv, Q., Liang, J., Wang, Y., Sun, D., Pasquier, T., Han, X., 2024. KAIROS: Practical intrusion detection and investigation using whole-system provenance. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 3533–3551.
- Defferrard, M., Bresson, X., Vandergheynst, P., 2016. Convolutional neural networks on graphs with fast localized spectral filtering. Adv. Neural Inf. Process. Syst. 29, 3844–3852.
- Ding, H., Zhai, J., Nan, Y., Ma, S., 2023. AIRTAG: Towards Automated Attack Investigation by Unsupervised Learning with Log Texts. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 373–390.
- Gehani, A., Tariq, D., 2012. SPADE: Support for provenance auditing in distributed environments. In: ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing. Springer, pp. 101–120.
- Han, M., Kim, H., Gu, G., Park, K., Han, W.-S., 2019. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In: Proceedings of the 2019 International Conference on Management of Data. pp. 1429–1446.
- Han, X., Pasquier, T., Bates, A., Mickens, J., Seltzer, M., 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats. arXiv Preprint arXiv: 2001.01525.
- Han, X., Yu, X., Pasquier, T., Li, D., Rhee, J., Mickens, J., Seltzer, M., Chen, H., 2021. (SIGL): Securing software installations through deep graph learning. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 2345–2362.
- Hassan, W.U., Nouredine, M.A., Datta, P., Bates, A., 2020. OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis. In: Network and Distributed System Security Symposium.

- Husari, G., Al-Shaer, E., Ahmed, M., Chu, B., Niu, X., 2017. Ttpdrill: Automatic and accurate extraction of threat actions from unstructured text of cti sources. In: Proceedings of the 33rd Annual Computer Security Applications Conference. pp. 103–115.
- Inam, M.A., Chen, Y., Goyal, A., Liu, J., Mink, J., Michael, N., Gaur, S., Bates, A., Hassan, W.U., 2023. Sok: History is a vast early warning system: Auditing the provenance of system intrusions. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 2620–2638.
- Ji, Y., Lee, S., Lee, W., 2016. Recprov: Towards provenance-aware user space record and replay. In: Provenance and Annotation of Data and Processes: 6th International Provenance and Annotation Workshop, IPAW 2016, McLean, VA, USA, June 7–8, 2016, Proceedings 6. Springer, pp. 3–15.
- Jiang, G., Mohandas, R., Leathery, J., Berry, A., Galang, L., 2017. CVE-2017-0199. <https://www.mandiant.com/resources/blog/cve-2017-0199-hita-handler>. (Accessed 10 October 2024).
- Kapoor, M., Melton, J., Ridenhour, M., Krishnan, S., Moyer, T., 2021. PROV-GEM: Automated provenance analysis framework using graph embeddings. In: 2021 20th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 1720–1727.
- Le, Q., Mikolov, T., 2014. Distributed representations of sentences and documents. In: International Conference on Machine Learning. PMLR, pp. 1188–1196.
- Le, V.H., Zhang, H., 2023. Log parsing with prompt-based few-shot learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 2438–2449.
- Lee, K.H., Zhang, X., Xu, D., 2013. High accuracy attack provenance via binary-based execution partition. In: NDSS, vol. 16.
- Li, L., Chen, W., 2024. ConGraph: Advanced persistent threat detection method based on provenance graph combined with process context in cyber-physical system environment. *Electronics* 13 (5), 945.
- Liu, H., Wang, T., Li, Y., Lang, C., Jin, Y., Ling, H., 2023. Joint graph learning and matching for semantic feature correspondence. *Pattern Recognit.* 134, 109059.
- Ma, S., Zhai, J., Kwon, Y., Lee, K.H., Zhang, X., Ciocarlie, G., Gehani, A., Yegneswaran, V., Xu, D., Jha, S., 2018. {Kernel – Supported} {Cost – Effective} audit logging for causality tracking. In: 2018 USENIX Annual Technical Conference. USENIX ATC 18, pp. 241–254.
- Ma, S., Zhai, J., Wang, F., Lee, K.H., Zhang, X., Xu, D., 2017. MPI: Multiple perspective attack investigation with semantic aware execution partitioning. In: 26th USENIX Security Symposium. USENIX Security 17, pp. 1111–1128.
- Mawhirter, D., Reinehr, S., Han, W., Fields, N., Claver, M., Holmes, C., McClurg, J., Liu, T., Wu, B., 2021. Dryadic: Flexible and fast graph pattern matching at scale. In: 2021 30th International Conference on Parallel Architectures and Compilation Techniques. PACT, IEEE, pp. 289–303.
- Mawhirter, D., Wu, B., 2019. Automine: harmonizing high-level abstraction and high performance for graph mining. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. pp. 509–523.
- Narayanan, A., Chandramohan, M., Venkatesan, R., Chen, L., Liu, Y., Jaiswal, S., 2017. Graph2vec: Learning distributed representations of graphs. *arXiv Preprint arXiv:1707.05005*.
- National Vulnerability Database (NVD), 2016. CVE-2016-5195. <https://nvd.nist.gov/vuln/detail/CVE-2016-5195>. (Accessed 10 October 2024).
- National Vulnerability Database (NVD), 2021. CVE-2021-3156. <https://nvd.nist.gov/vuln/detail/CVE-2021-3156>. (Accessed 10 October 2024).
- Navarro, J., Deruyver, A., Parrend, P., 2018. A systematic survey on multi-step attack detection. *Comput. Secur.* 76, 214–249.
- Neuhaus, M., Riesen, K., Bunke, H., 2006. Fast suboptimal algorithms for the computation of graph edit distance. In: Structural, Syntactic, and Statistical Pattern Recognition: Joint IAPR International Workshops, SSPR 2006 and SPR 2006, Hong Kong, China, August 17–19, 2006. Proceedings. In: Lecture Notes in Computer Science, vol. 4109, Springer, Berlin, Heidelberg, pp. 163–172.
- Pasquier, T., Han, X., Moyer, T., Bates, A., Hermant, O., Eysers, D., Bacon, J., Seltzer, M., 2018. Runtime analysis of whole-system provenance. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 1601–1616.
- Pohly, D.J., McLaughlin, S., McDaniel, P., Butler, K., 2012. Hi-fi: collecting high-fidelity whole-system provenance. In: Proceedings of the 28th Annual Computer Security Applications Conference. pp. 259–268.
- Qaiser, S., Ali, R., 2018. Text mining: use of TF-IDF to examine the relevance of words to documents. *Int. J. Comput. Appl.* 181 (1), 25–29.
- Rehman, M.U., Ahmadi, H., Hassan, W.U., 2024. FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, 139–139.
- Rong, X., 2014. Word2vec parameter learning explained. *arXiv Preprint arXiv:1411.2738*, URL <https://arxiv.org/abs/1411.2738>.
- Sinha, J., Manollas, M., 2020. Efficient deep CNN-bilstm model for network intrusion detection. In: Proceedings of the 2020 3rd International Conference on Artificial Intelligence and Pattern Recognition. pp. 223–231.
- Socher, R., Chen, D., Manning, C.D., Ng, A., 2013. Reasoning with neural tensor networks for knowledge base completion. In: Advances in Neural Information Processing Systems, vol. 26.
- Tang, Y., Li, D., Li, Z., Zhang, M., Jee, K., Xiao, X., Wu, Z., Rhee, J., Xu, F., Li, Q., 2018. Nodemerger: Template based efficient data reduction for big-data causality analysis. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 1324–1337.
- Trellix Research Team, 2023. CVE-2023-38831: Navigating the Threat Landscape of the Latest Security Vulnerability. <https://www.trellix.com/blogs/research/cve-2023-38831-navigating-the-threat-landscape-of-the-latest-security-vulnerability/>. (Accessed 10 October 2024).
- Trend Micro Research Team, 2016. Exploit Kits 2015: Flash Bugs, Malvertising Dominate. https://www.trendmicro.com/en_us/research/16/c/exploit-kits-2015-flash-bugs-compromised-sites-malvertising-dominate.html. (Accessed 10 October 2024).
- Wang, S., Chen, Z., Yu, X., Li, D., Ni, J., Tang, L.-A., Gui, J., Li, Z., Chen, H., Yu, P.S., 2019. Heterogeneous graph matching networks. *arXiv Preprint arXiv:1910.08074*.
- Wang, Q., Hassan, W.U., Li, D., Jee, K., Yu, X., Zou, K., Rhee, J., Chen, Z., Cheng, W., Gunter, C.A., et al., 2020. You are what you do: Hunting stealthy malware via data provenance analysis. In: NDSS.
- Wei, R., Cai, L., Zhao, L., Yu, A., Meng, D., 2021. Deephunter: A graph neural network based approach for robust cyber threat hunting. In: Security and Privacy in Communication Networks: 17th EAI International Conference, SecureComm 2021, Virtual Event, September 6–9, 2021, Proceedings, Part I 17. Springer, pp. 3–24.
- Xu, Z., Wu, Z., Li, Z., Jee, K., Rhee, J., Xiao, X., Xu, F., Wang, H., Jiang, G., 2016. High fidelity data reduction for big data security dependency analyses. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 504–516.
- Yang, F., Xu, J., Xiong, C., Li, Z., Zhang, K., 2023. {PROGRAPHER}: An anomaly detection system based on provenance graph embedding. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 4355–4372.
- Ying, J., Zhu, T., Cheng, W., Yuan, Q., Ma, M., Xiong, C., Chen, T., Lv, M., Chen, Y., 2024. SPARSE: Semantic tracking and path analysis for attack investigation in real-time. *arXiv Preprint arXiv:2405.02629*.
- Yue, H., Li, T., Wu, D., Zhang, R., Yang, Z., 2024. Detecting APT attacks using an attack intent-driven and sequence-based learning approach. *Comput. Secur.* 140, 103748.
- Zengy, J., Wang, X., Liu, J., Chen, Y., Liang, Z., Chua, T.-S., Chua, Z.L., 2022. Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 489–506.